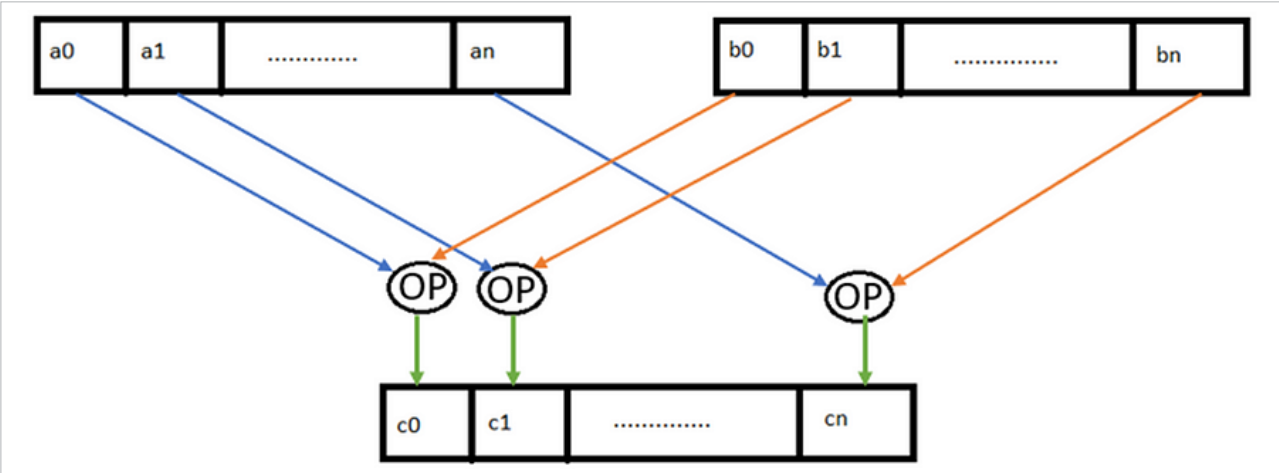

Figure 2: Simple depiction of SIMD execution

at runtime, while maintaining efficient vectorized operation.

Where does Java Vector API fit in?

Instruction-level and bit-level parallelism are inherent to the hardware. Their efficiency depends on factors like the sophistication of the branch prediction algorithm and the design of the pipelining process in CPU microarchitecture. These hardware-specific features vary from one platform to another and are not typically accessible through open APIs for developers.

Data and task parallelism are accessible within the programming language, and developers can use specific APIs to leverage these forms of parallelism. Task parallelism can be achieved in Java using the concurrency and multi-threading APIs in the *java.util.concurrent* package of Java.

Application-level performance in multi-core systems can be optimised through data parallelism using the single instruction, multiple data or SIMD approach. There are platform-specific instructions called SIMD instructions that can enable data parallelism. As these are different for different hardware, they become tedious to keep track of. Java Vector API sweeps in and acts like a diplomat between the underlying platform-specific code and the high-level Java code, which is easy to understand and implement, keeping Java's portability philosophy of 'write once and run anywhere' intact.

Data vectorization using SIMD

SIMD is a computing method using which a single operation, such as an arithmetic operation or a complex operation like fused multiply-add (FMA), is applied across two vectors (sets of data points and not single data points). SIMD instructions of the underlying platform are used to achieve data parallelism. Here the vector registers, supported by the underlying platform, are utilised to realise data parallelism.

In a sequential or scalar approach, each data element is processed one at a time. While, in a single instruction cycle, SIMD can execute operations on multiple data elements simultaneously, improving efficiency and reducing costs.

An important aspect of SIMD is data independency — one iteration should not affect the outcome of another and the data being operated on should be homogenous in nature.

Components of Java Vector API code

Java Vector API provides platform-agnostic ways for Java developers to write data parallel algorithms without necessarily knowing the specifics of that hardware. It is the JIT compiler's responsibility to then translate the high-level Java Vector API code into appropriate SIMD instructions to leverage the hardware capabilities at runtime. The constructs in Java Vector APIs that developers can use to write data parallel algorithms are discussed below.

- ***Vector shape***

Vector shape refers to the bit width a specific platform can support. Each platform has its limitations regarding how many bits it can handle in a single vector operation, based on its support of vector registers. This bit width of the vector registers determines the amount of data that can be processed in parallel in a single vector instruction. For example, a platform supporting a 512-bit width register can handle more data elements processing simultaneously compared to a platform with a lower bit width support. The API to get the underlying platform's default shape supported is *VectorShape.preferredShape()*. It can be either 64, 128, 256 or 512-bit width based on the underlying platform.

- ***Vector species***

Vector species is a combination of a data type (like int, float, etc) and the vector shape (bit width). Each element